System Design Document

DAG-Based Workflow Orchestrator

Problem	Problem 2: DAG-Based Workflow Orchestrator
Stack	Node.js + TypeScript | PostgreSQL | Fastify
Scope	Single-process | 10 concurrent workers | 500-task DAG deliverable
Doc version	1.0 — Post adversarial design review, all 12 decisions locked

1. Overview & Scope

This document is the complete system design for a production-grade DAG-based workflow orchestrator built from scratch in Node.js + TypeScript. It covers every major component: the submission API, cycle detection, persistent storage schema, the in-memory priority queue, the ready-set scheduler, the bounded worker pool, retry mechanics, the cron scheduler, crash recovery, and idempotency semantics. For every structural choice, this document records the alternatives considered and the engineering rationale for the decision made.

> **Evaluation rubric reminder:** Correctness under edge cases (30%) | Data structure design (25%) | Concurrency, durability, crash recovery (20%) | Code quality & tests (15%) | Documentation & trade-off defence (10%)

1.1 What we are NOT building

- Any workflow library wrapper (Temporal, BullMQ, Agenda, node-cron, Bree — all forbidden)
- A distributed multi-node system (single-process with optional multi-process bonus described in §10)
- A workflow template registry (cron schedules store inline JSONB; v2 evolution described in §9)

1.2 Deliverables checklist

Deliverable	Section
500-task diamond DAG under 10 concurrent workers	§6, §7
kill -9 mid-execution recovery test	§8
Cron parser — 15+ expressions including 0 0 29 2 *	§9
Retry storm — 100 tasks x 5 failures	§5.3
Architecture diagram	§2
Design doc — data structures, trade-offs, production hardening	This document

2. Architecture Overview

The system is a single Node.js process with five logical subsystems communicating through shared in-memory structures and PostgreSQL as the source of truth for durable state. The diagram below shows the data flow from workflow submission through execution and recovery.

2.1 Key design principles

• **PostgreSQL is the system of record.** All durable state lives there. The in-memory heap is a scheduling cache; losing it is safe because tasks in READY state are reloaded on startup.

• **Two writes per transition.** Write the new state before execution, write the final result after. Crash between the two → task is reclaimed by the reaper.

• **DB-atomic ready-set.** pending_deps decrements happen inside UPDATE ... RETURNING. No application-level TOCTOU race possible.

• **No external message broker.** PostgreSQL SKIP LOCKED serves as the work queue primitive, eliminating an operational dependency.

• **Cooperative semantics everywhere.** Cancellation, shutdown, and timeout all rely on check-points and Promise.race rather than forceful termination, keeping handler side effects auditable.

3. Data Model & Schema

3.1 Design decision: PostgreSQL vs MySQL

CHOSEN	**PostgreSQL — SELECT ... FOR UPDATE SKIP LOCKED, atomic UPDATE...RETURNING, JSONB, advisory locks**
ALT	MySQL 8+ — SKIP LOCKED available but gap-lock semantics are less predictable; RETURNING not natively supported pre-8.0
ALT	SQLite — forbidden by spec for this problem; no row-level locking
ALT	Redis + Lua — fast but loses durability without AOF/RDB tuning; adds an operational dependency

PostgreSQL's SKIP LOCKED is the core primitive that allows N workers to each claim a distinct READY task atomically without coordination logic in the application layer. RETURNING on UPDATE is used for the ready-set decrement: the worker that observes pending_deps = 0 in the returned row is the unique enqueuer of that child task. JSONB stores handler input and workflow definition inline without a separate schema migration per workflow type.

3.2 Schema

workflows

Column	Type	Description
`id`	`UUID PK`	Stable workflow identifier, v4 random
`status`	`TEXT`	PENDING | RUNNING | COMPLETED | FAILED | CANCELLING | CANCELLED
`created_at`	`TIMESTAMPTZ`	Submission timestamp
`updated_at`	`TIMESTAMPTZ`	Last status transition timestamp

tasks

Column	Type	Description
`id`	`UUID PK`	Task identifier — caller-supplied or auto-generated
`workflow_id`	`UUID FK`	Parent workflow
`handler_name`	`TEXT`	Key into the in-process handler registry
`input`	`JSONB`	Opaque input passed to the handler
`status`	`TEXT`	PENDING | READY | RUNNING | COMPLETED | FAILED | CANCELLED
`priority`	`INT DEFAULT 0`	Higher = more urgent; heap key is -priority
`scheduled_at`	`TIMESTAMPTZ`	Earliest execution time (used for backoff scheduling)
`submission_order`	`BIGINT`	Global monotonic sequence for tie-breaking in heap
`attempts`	`INT DEFAULT 0`	Number of execution attempts so far
`max_attempts`	`INT`	From retryPolicy.maxAttempts
`backoff_base_ms`	`INT`	Backoff base in ms
`backoff_cap_ms`	`INT`	Backoff cap in ms
`last_sleep_ms`	`INT DEFAULT 0`	Previous sleep duration — required for decorrelated jitter
`timeout_ms`	`INT`	Hard execution timeout per attempt
`pending_deps`	`INT DEFAULT 0`	Count of incomplete dependsOn tasks; 0 = READY
`last_heartbeat_at`	`TIMESTAMPTZ`	Updated every ~5s while RUNNING; reaper uses this

Column	Type	Description
task_run_id	UUID	Attempt-scoped UUID; exposed as idempotencyKey to handler
error	TEXT	Last failure message
completed_at	TIMESTAMPTZ	Terminal state timestamp

task_dependencies

Column	Type	Description
task_id	UUID FK	Dependent task
depends_on_task_id	UUID FK	Prerequisite task

schedules

Column	Type	Description
id	UUID PK	Schedule identifier
cron_expression	TEXT	Standard 5-field cron e.g. */15 * * * *
timezone	TEXT DEFAULT 'UTC'	IANA timezone e.g. America/New_York
workflow_definition	JSONB	Full workflow definition cloned on each fire
next_fire_at	TIMESTAMPTZ	Pre-computed next fire time; updated after each fire
enabled	BOOLEAN DEFAULT TRUE	Soft-disable without deleting
last_fired_at	TIMESTAMPTZ NULL	Audit trail

3.3 Key indices

- tasks(workflow_id, status) — bulk status queries for GET /workflows/:id
- tasks(status, scheduled_at, priority) — scheduler poll for READY tasks
- tasks(workflow_id, status) WHERE status='RUNNING' — partial index for reaper
- task_dependencies(depends_on_task_id) — child lookup on parent completion
- schedules(next_fire_at) WHERE enabled=TRUE — cron tick poll

---

4. Workflow Submission & Cycle Detection

4.1 Submission flow

- **Step V:** lidate JSON shape (required fields, handler names registered)
- **Step R:** n DFS three-coloring cycle detection on the dependsOn adjacency — reject with cycle path if found
- **Step C:** mpute pending_deps for each task = len(dependsOn)
- **Step B:** GIN transaction: insert workflow row (PENDING), insert all task rows, insert task_dependency rows
- **Step M:** rk tasks with pending_deps = 0 as READY, transition workflow to RUNNING
- **Step C:** MMIT — response returns workflow id

4.2 Cycle detection algorithm

CHOSEN	**DFS Three-Coloring (WHITE / GRAY / BLACK) — surfaces exact cycle path**
ALT	Kahn's Algorithm (BFS topological sort) — correct, O(V+E), but only returns boolean; no cycle path for diagnostics
ALT	Floyd-Warshall transitive closure — O(V^3) space and time; overkill for DAG validation at submission

ALT	Union-Find — detects cycles in undirected graphs; not directly applicable to directed DAGs without modification

Three-coloring assigns WHITE (unseen), GRAY (currently on the DFS call stack), and BLACK (fully explored) to each node. A back-edge — an edge to a GRAY node — proves a cycle. The algorithm records the full path from the GRAY ancestor to the current node, so the rejection error is: 'Cycle detected: task-A → task-B → task-C → task-A'. Complexity is O(V + E) — identical to Kahn's — but the diagnostic value is materially superior for a caller debugging a malformed DAG.

```
function dfs(node, color, adj, path): string[] | null {
color[node] = GRAY; path.push(node);
for (const neighbor of adj[node]) {
if (color[neighbor] === GRAY) return [...path, neighbor]; // back-edge
if (color[neighbor] === WHITE) {
const cycle = dfs(neighbor, color, adj, path);
if (cycle) return cycle;
}
}
color[node] = BLACK; path.pop(); return null;
}
```

4.3 Edge cases tested

- Self-loop: task A dependsOn: ['A']
- Diamond: A→B, A→C, B→D, C→D — valid, no cycle; both B and C must complete before D
- Disconnected components: separate subgraphs within one workflow submission
- Empty dependsOn: all tasks READY on submission
- Single task workflow: trivially valid

---

5. Retry Mechanics & Backoff

5.1 Jitter strategy selection

CHOSEN	**Decorrelated jitter: sleep = min(cap, random(base, prev_sleep × 3))**
ALT	Full jitter: random(0, min(cap, base × 2^attempt)) — excellent spread but can retry near-instantly (sleep near 0)
ALT	Equal jitter: min(cap, base×2^attempt)/2 + random(0, half) — bounded floor, less variance, no memory of prev sleep
ALT	Fixed exponential (no jitter) — thundering herd on correlated failures; unacceptable for retry storm scenario
ALT	Constant backoff — no backoff growth; violates the spirit of retryPolicy.cap

Decorrelated jitter (originally described in the AWS Architecture Blog) self-adapts: if a previous retry slept for a short interval, the next range is narrow. If it slept long, the range widens. This naturally spreads retries across time without requiring knowledge of the attempt count or a shared coordination structure. The cost is one extra column (last_sleep_ms) per task. The 100-task × 5-failure retry storm test specifically exercises this: all 100 tasks fail simultaneously on attempt 1. Without jitter, all 100 would be re-enqueued at identical timestamps, causing a second thundering herd.

5.2 Backoff formula

```
// On task failure, before re-scheduling:
```

```
const prevSleep = task.last_sleep_ms || retryPolicy.backoffBase;
const rawSleep = Math.random() * (prevSleep * 3 - retryPolicy.backoffBase)
+ retryPolicy.backoffBase;
const sleep = Math.min(retryPolicy.backoffCap, rawSleep);
const scheduledAt = Date.now() + sleep;
// Persist: UPDATE tasks SET last_sleep_ms=$1, scheduled_at=$2,
// attempts=attempts+1, status='PENDING' WHERE id=$3
```

5.3 Retry state machine

A FAILED attempt increments attempts. If attempts < max_attempts, the task transitions back to PENDING with a future scheduled_at. The scheduler will only dequeue it once scheduled_at <= now. If attempts = max_attempts, the task transitions to terminal FAILED and the workflow engine checks whether all sibling tasks have completed — if any are terminal FAILED the workflow transitions to FAILED.

5.4 Timeout enforcement

CHOSEN	**Hard timeout: Promise.race([handler(input, {signal}), timeoutReject(timeoutMs)])**
ALT	Soft timeout (log + continue) — does not enforce timeoutMs; not defensible
ALT	process.kill() — cannot target a single async operation in single-process model
ALT	Worker threads with terminate() — adds complexity; overkill for single-process scope

AbortSignal is passed to every handler invocation. Well-behaved handlers propagate it to downstream I/O (fetch({signal}), pg client query cancellation). On timeout, the rejection is caught, counted as a failure, and the retry policy is evaluated. The timeout clock starts when the handler function is invoked, not when the task enters the queue.

---

6. In-Memory Priority Queue

6.1 Data structure selection

CHOSEN	**Binary min-heap keyed on (-priority, scheduled_at, submission_order)**
ALT	Array.sort() per dequeue — O(n log n) per operation; explicitly forbidden by spec
ALT	Sorted doubly-linked list — O(n) insert, O(1) extract-min; poor insert performance under load
ALT	Fibonacci heap — O(1) amortised insert, O(log n) extract; high constant factors, complex implementation, not worth it at this scale
ALT	Skip list — probabilistic O(log n); suitable but harder to implement than a heap and offers no advantage here
ALT	Map-as-sorted-structure — forbidden by spec

Binary min-heap gives O(log n) insert and O(log n) extract-min with very low constant factors and a simple array representation. The heap array stores TaskHeapEntry objects containing the compound sort key. Sift-up and sift-down are implemented from scratch as plain index arithmetic on a JavaScript array.

6.2 Compound key semantics

```
type TaskHeapEntry = {
taskId: string;
priority: number; // negated → higher priority = smaller key
scheduledAt: number; // epoch ms
```

```
submissionOrder: bigint; // global monotonic sequence from DB
};

function compare(a: TaskHeapEntry, b: TaskHeapEntry): number {
if (-a.priority !== -b.priority) return -a.priority - (-b.priority);
if (a.scheduledAt !== b.scheduledAt) return a.scheduledAt - b.scheduledAt;
return a.submissionOrder < b.submissionOrder ? -1 : 1;
}
```

6.3 Heap operations

- **insert(entry)** — O(log n) — Append to tail, sift-up
- **extractMin()** — O(log n) — Swap root with tail, pop tail, sift-down
- **peek()** — O(1) — Read heap[0]
- **size()** — O(1) — Array length

6.4 Heap vs DB interaction

The heap is a scheduling cache, not the source of truth. On startup, all tasks with status=READY and scheduled_at <= now are loaded from PostgreSQL and inserted into the heap. This means the heap can be lost on crash (process kill) without data loss — recovery simply reloads READY tasks from the DB. Newly READY tasks (pending_deps hits 0) are inserted into the heap immediately in the same transaction callback that updates the DB.

---

7. Ready-Set Scheduler & Worker Pool

7.1 The ready-set race problem

Consider task D that dependsOn: [B, C]. Tasks B and C complete concurrently on two different event loop ticks (or, in a multi-process deployment, on two different nodes). Both read pending_deps = 1 and attempt to decrement it. An application-level read-modify-write would have both read 1, both write 0, and both enqueue D — resulting in double execution. The correct approach uses a single atomic DB operation.

7.2 Atomic decrement

CHOSEN	**UPDATE tasks SET pending_deps=pending_deps-1 WHERE id=$1 RETURNING pending_deps**
ALT	Application-level read-modify-write — TOCTOU race; double-enqueue possible
ALT	In-memory adjacency + Mutex — fast but lost on crash; recovery must replay from DB anyway
ALT	Advisory lock per task — correct but adds lock acquisition overhead per completion

```
// In the task completion handler:
const childIds = await db.query(
`SELECT task_id FROM task_dependencies WHERE depends_on_task_id = $1`,
[completedTaskId]
);
for (const { task_id } of childIds.rows) {
const result = await db.query(
`UPDATE tasks
SET pending_deps = pending_deps - 1
WHERE id = $1 AND status = 'PENDING'
RETURNING pending_deps, id`,
[task_id]
);
if (result.rows[0]?.pending_deps === 0) {
```

```
await db.query(
`UPDATE tasks SET status='READY', scheduled_at=NOW() WHERE id=$1`, [task_id]
);
heap.insert(buildHeapEntry(result.rows[0]));
}
}
```

7.3 Worker pool — bounded semaphore

CHOSEN	**Bounded semaphore: integer counter + Promise-based waiter queue**
ALT	p-limit / async-pool / bottleneck — forbidden by spec; would hide implementation
ALT	Worker threads — adds IPC serialisation cost; single-process scope doesn't need parallelism
ALT	Cluster module — multi-process; out of scope for the single-process baseline

```
class BoundedSemaphore {
private count: number;
private waiters: Array<() => void> = [];
constructor(private readonly limit: number) { this.count = limit; }

async acquire(): Promise<void> {
if (this.count > 0) { this.count--; return; }
await new Promise<void>(resolve => this.waiters.push(resolve));
}

release(): void {
const next = this.waiters.shift();
if (next) { next(); } else { this.count++; }
}
}
```

7.4 Worker execution loop

```
async function runTask(task: Task): Promise<void> {
await semaphore.acquire();
try {
// Write 1: mark RUNNING before touching handler
const runId = randomUUID();
await db.query(`UPDATE tasks SET status='RUNNING', task_run_id=$1,
last_heartbeat_at=NOW(), attempts=attempts+1 WHERE id=$2`,
[runId, task.id]);

const heartbeat = setInterval(() =>
db.query(`UPDATE tasks SET last_heartbeat_at=NOW() WHERE id=$1`, [task.id]),
HEARTBEAT_INTERVAL_MS);

const ac = new AbortController();
const timer = setTimeout(() => ac.abort(), task.timeoutMs);

let outcome: 'completed' | 'failed' = 'failed';
let error: string | undefined;
try {
const handler = registry.get(task.handlerName)!;
await Promise.race([
handler(task.input, { signal: ac.signal, idempotencyKey: runId,
workflowId: task.workflowId, taskId: task.id,
attempt: task.attempts }),
new Promise((_, reject) =>
ac.signal.addEventListener('abort', () => reject(new Error('timeout'))))
```

```
]);
outcome = 'completed';
} catch (e) { error = String(e); }
finally { clearTimeout(timer); clearInterval(heartbeat); }

// Write 2: persist final state
await persistOutcome(task, outcome, error);
} finally {
semaphore.release();
}
}
```

---

8. Crash Recovery & Durability

8.1 Persistence strategy

CHOSEN	**Strict: two DB writes per state transition + periodic heartbeat while RUNNING**
ALT	One write per transition — cannot distinguish completed vs crashed RUNNING without heartbeat; still needs heartbeat anyway
ALT	Event sourcing / append-only log — maximum auditability but adds read complexity to reconstruct current state
ALT	Optimistic single write with CAS version column — reduces writes but adds conflict-retry logic

Write 1 (before execution): status=RUNNING, task_run_id=, last_heartbeat_at=NOW(). Write 2 (after execution): status=COMPLETED or FAILED, completed_at=NOW(). A process crash between writes leaves the task in RUNNING with a stale heartbeat. The reaper identifies this and re-enqueues the task.

8.2 Heartbeat & reaper

While executing, a worker updates last_heartbeat_at every HEARTBEAT_INTERVAL_MS (default 5s). The reaper runs at startup and then every REAPER_POLL_MS (default 30s). It queries:

```
SELECT * FROM tasks
WHERE status = 'RUNNING'
AND last_heartbeat_at < NOW() - INTERVAL '${LEASE_TIMEOUT_MS} milliseconds'
AND attempts < max_attempts;
-- LEASE_TIMEOUT_MS default: 3 × HEARTBEAT_INTERVAL_MS = 15s
```

For each reclaimed task: increment attempts, apply decorrelated jitter backoff, set status=PENDING with future scheduled_at. If attempts >= max_attempts, mark FAILED.

8.3 Startup recovery sequence

- Run reaper: reclaim all stale RUNNING tasks
- Load all READY tasks into the in-memory heap
- Resume the scheduler loop
- Fire any cron schedules whose next_fire_at is in the past

8.4 kill -9 test protocol

```
// Test: spawn process, submit 10 tasks, kill -9 when 3 are RUNNING,
// restart, assert all 10 eventually reach COMPLETED or terminal FAILED

it('recovers from kill -9 mid-execution', async () => {
const proc = spawn('node', ['dist/server.js']);
```

```
await waitForHealth(proc);
const { workflowId } = await submitWorkflow(10_tasks);
await waitForNRunning(workflowId, 3);
proc.kill('SIGKILL'); // kill -9
const proc2 = spawn('node', ['dist/server.js']);
await waitForHealth(proc2);
await waitForTerminal(workflowId, timeout: 60_000);
const wf = await getWorkflow(workflowId);
expect(wf.tasks.every(t => ['COMPLETED','FAILED'].includes(t.status))).toBe(true);
// No task left in RUNNING state
expect(wf.tasks.filter(t => t.status === 'RUNNING')).toHaveLength(0);
});
```

---

9. Cron Scheduler

9.1 Timezone strategy

CHOSEN	**Timezone-aware: IANA zone per schedule (default UTC); skip DST gap; fire once on fold (first occurrence)**
ALT	UTC-only — eliminates DST boundaries; spec explicitly evaluates DST correctness; this is avoidance not a strategy
ALT	Fixed UTC offset (e.g. +05:30) — correct for non-DST zones but wrong for zones with DST (US, EU)
ALT	Moment-timezone — correct but adds a dependency; Intl.DateTimeFormat is native and sufficient

Each schedule row stores an IANA timezone string (e.g. 'America/New_York'). next_fire_at is always stored in UTC. The cron parser computes candidate fire times in local wall-clock time and converts to UTC using Intl.DateTimeFormat offset resolution. DST gap (spring forward, e.g. 2:00AM–3:00AM doesn't exist): if computed fire falls in the gap, advance to the first valid moment post-gap. DST fold (fall back, e.g. 2:00AM–3:00AM exists twice): fire on the first occurrence (standard-time offset); skip the second occurrence.

9.2 Cron expression grammar

Standard 5-field: minute hour day-of-month month day-of-week

```
Field Range Supported syntax
██████████ ███████ █████████████████████████████████████████████████████
minute 0-59 * */n n n-m n,m,k
hour 0-23 * */n n n-m n,m,k
day-of-month 1-31 * */n n n-m n,m,k (L = last day: not required)
month 1-12 * */n n n-m n,m,k
day-of-week 0-6 * */n n n-m n,m,k (0=Sun, 6=Sat)
Examples:
*/15 * * * * every 15 minutes
0 9 * * 1-5 weekdays at 09:00
0 0 29 2 * Feb 29 midnight (leap year only → next: 2028-02-29 UTC)
0 0 1 1 * Jan 1 midnight
30 8,12,17 * * * 08:30, 12:30, 17:30 daily
```

9.3 next-fire algorithm

```
function nextFire(expr: CronExpr, after: Date, tz: string): Date {
let candidate = new Date(after.getTime() + 60_000); // advance 1 minute
candidate.setSeconds(0, 0);
```

```
for (let i = 0; i < 366 * 24 * 60; i++) { // max 1 year search
const local = toLocal(candidate, tz); // wall-clock fields in tz
if (matches(expr, local)) {
const resolved = resolveAmbiguity(candidate, tz); // DST fold/gap
return resolved;
}
candidate = new Date(candidate.getTime() + 60_000);
}
throw new Error('No fire time found within 1 year');
}
```

9.4 Schedule fire loop

```
// Runs every CRON_TICK_MS (default 10s)
async function cronTick(): Promise<void> {
const due = await db.query(
`SELECT * FROM schedules WHERE enabled=TRUE AND next_fire_at <= NOW()`
);
for (const schedule of due.rows) {
// Deep-clone inline definition, submit as new workflow
await submitWorkflow(JSON.parse(JSON.stringify(schedule.workflow_definition)));
const next = nextFire(parse(schedule.cron_expression),
new Date(), schedule.timezone);
await db.query(
`UPDATE schedules SET next_fire_at=$1, last_fired_at=NOW() WHERE id=$2`,
[next, schedule.id]
);
}
}
```

---

10. Idempotency, Cancel & Workflow Lifecycle

10.1 Idempotency primitives

CHOSEN	**Expose attempt-scoped task_run_id (UUID) as context.idempotencyKey to every handler**
ALT	Framework-managed idempotency store — framework cannot know the handler's external targets; over-engineering
ALT	workflow_id + task_id as key — stable across attempts; wrong for idempotency (same key reused on retry)
ALT	No primitive — handler authors have no dedup anchor; at-least-once becomes practically de-facto at-most-once by accident

task_run_id is generated fresh on every attempt (new UUID each time status transitions to RUNNING). It is written to the DB before handler invocation (Write 1), so it is stable for the duration of the attempt even if the handler is retried internally. Handler authors use it as a dedup key:

```
// Handler example — idempotent external API call
async function sendShipmentNotification(input, { idempotencyKey, signal }) {
await fetch('https://notify.example.com/send', {
method: 'POST',
headers: { 'Idempotency-Key': idempotencyKey },
body: JSON.stringify(input),
signal,
```

```
});
}

// Handler example — idempotent DB write
async function createOrder(input, { idempotencyKey }) {
await db.query(
`INSERT INTO orders (id, ...) VALUES ($1, ...) ON CONFLICT (id) DO NOTHING`,
[idempotencyKey, ...]
);
}
```

10.2 Cancel semantics

CHOSEN	**Cooperative cancel: mark workflow CANCELLING; running tasks complete naturally; dependents never enqueued**
ALT	Preemptive abort (ac.abort() on in-flight handlers) — handlers must be abort-aware; side effects may be partial
ALT	Immediate CANCELLED + DB rollback — no rollback primitive defined; external side effects cannot be rolled back
ALT	Poison-pill message to workers — adds coordination complexity in single-process model

State machine: workflow transitions RUNNING → CANCELLING on POST /workflows/:id/cancel. Workers check workflow status before dequeuing any READY task: if CANCELLING, the task is marked CANCELLED without execution. Already-running tasks execute to completion (or timeout). When all tasks reach a terminal state (COMPLETED, FAILED, CANCELLED), the workflow transitions to CANCELLED.

10.3 Workflow state machine

```
■■■■■■■■■■■■
submit ■ ■
■■■■■■■■■■■■ RUNNING ■■■■■■ all tasks terminal ■■■■■■ COMPLETED
■ ■■■■■■ any task FAILED ■■■■■■■■■■ FAILED
■ ■■■■■■ cancel request ■■■■■■■■■■■ CANCELLING
■■■■■■■■■■■■ ■
■ all tasks drained
▼
CANCELLED
```

10.4 Task state machine

```
PENDING ■■[pending_deps=0]■■■ READY ■■[worker picks up]■■■ RUNNING
■
■■■■■■ success ■■■■■■ COMPLETED
■■■■■■ failure,
■ attempts<max ■ PENDING (backoff)
■■■■■■ failure,
■ attempts=max ■ FAILED
■■■■■■ timeout ■■■■■■■■■ same as failure
■■■■■■ wf CANCELLING ■■ CANCELLED
```

---

11. API Contract

11.1 POST /workflows

```
Request body:
{
"tasks": [
{
"id": "string (caller-supplied)",
"handler": "registered-handler-name",
"input": { ...arbitrary JSON... },
"dependsOn": ["task-id-1", "task-id-2"],
"retryPolicy": { "maxAttempts": 3, "backoffBase": 1000, "backoffCap": 30000 },
"timeoutMs": 5000,
"priority": 0
}
]
}
Response 201: { workflowId: string }
Response 422: { error: 'Cycle detected: A -> B -> C -> A' }
Response 422: { error: 'Unknown handler: foo' }
```

11.2 Other endpoints

- **GET /workflows/:id** — Returns workflow status + all task statuses. O(n) DB read, no joins beyond tasks table.
- **POST /workflows/:id/cancel** — Transitions workflow to CANCELLING if currently RUNNING. Idempotent: CANCELLING/CANCELLED → 200 with no-op.
- **POST /schedules** — Registers cron schedule. Computes and stores next_fire_at immediately. Returns schedule id.

---

12. Production Hardening (Out-of-Scope for Submission)

12.1 Multi-process workers

Move to a cluster model where N worker processes each run the scheduler loop. Task claiming is already correct via SKIP LOCKED. Add a distributed lease: each worker registers a row in a workers table with a heartbeat. A separate reaper process reclaims tasks whose worker_id references a dead worker. The ready-set DB-atomic decrement is already safe across processes.

12.2 Observability

- Structured JSON logging with trace_id, workflow_id, task_id on every log line
- Prometheus metrics: task_throughput_total, task_duration_p99, queue_depth, reaper_reclaimed_total
- OpenTelemetry spans per task execution for distributed tracing
- Dead-letter queue: tasks that exhaust retries written to a separate dead_letter_tasks table for inspection

12.3 Schema evolution

- Use a migration tool (e.g. node-pg-migrate) — no schema changes in application code
- Version the workflow_definition JSONB with a schema_version field for forward compatibility
- Named template registry (POST /templates) as a v2 evolution of inline cron definitions

12.4 Security

- Handler registry validates names against an allowlist at registration time, not at submission
- Input JSONB size limit (default 64KB) to prevent heap exhaustion via large payloads
- Rate limit POST /workflows per API key to prevent queue flooding

• Row-level security on workflows table if multi-tenant deployment

12.5 Performance

• Batch the pending_deps decrement: single UPDATE ... WHERE id = ANY($1) for all children of a completed task
• Partial index on tasks(status, scheduled_at) WHERE status IN ('READY','RUNNING') to keep index small
• Connection pooling via pgBouncer in transaction mode for multi-process deployment
• Heap warm-up: load top-K READY tasks on startup rather than full scan for large backlogs

---

13. Test Matrix

Category	Test	Validates
Cycle detection	`Self-loop: A→A`	DFS back-edge on first neighbor
Cycle detection	`Diamond: A→B,C; B,C→D`	Valid DAG, no false positive
Cycle detection	`Disconnected components`	Both subgraphs checked
Cycle detection	`Long chain + back-edge`	Full cycle path in error
Ready-set	`500-task diamond DAG`	Correct completion order, no double-exec
Ready-set	`Concurrent parent completion`	pending_deps hits 0 exactly once
Priority queue	`Priority ties → FIFO by submission_order`	Heap comparator
Priority queue	`Heap with 10K entries`	Extract-min correctness
Retry / backoff	`Retry storm: 100 tasks × 5 failures`	Decorrelated spread
Retry / backoff	`Backoff timestamps match formula`	last_sleep_ms progression
Timeout	`Handler exceeds timeoutMs`	Promise.race rejects, attempt++
Crash recovery	`kill -9 mid-execution`	All tasks terminal post-restart
Crash recovery	`Reaper reclaims stale RUNNING`	last_heartbeat check
Cancel	`Cancel while tasks running`	Running complete, dependents CANCELLED
Cancel	`Cancel already-cancelled`	Idempotent 200
Cron parser	`*/15 * * * *`	Every 15 min
Cron parser	`0 0 29 2 *`	Next: 2028-02-29 UTC
Cron parser	`0 2 * * * (DST spring forward)`	Gap skipped, fires post-gap
Cron parser	`0 1 * * * (DST fall back)`	Fires once on first occurrence
Cron parser	`15 expressions total`	Parser correctness suite
Idempotency	`task_run_id changes on retry`	New UUID per attempt
API	`Submit invalid handler name`	422 with clear message
API	`GET /workflows/:id mid-execution`	Correct per-task status